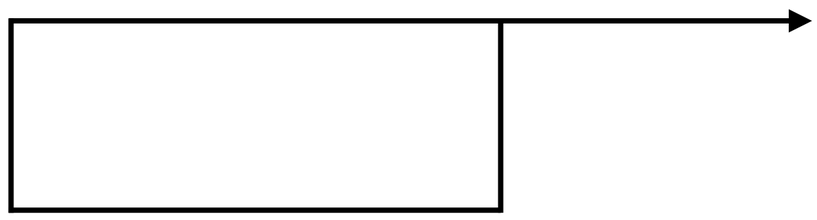
17. Physical Memory

Constructing a Memory Register

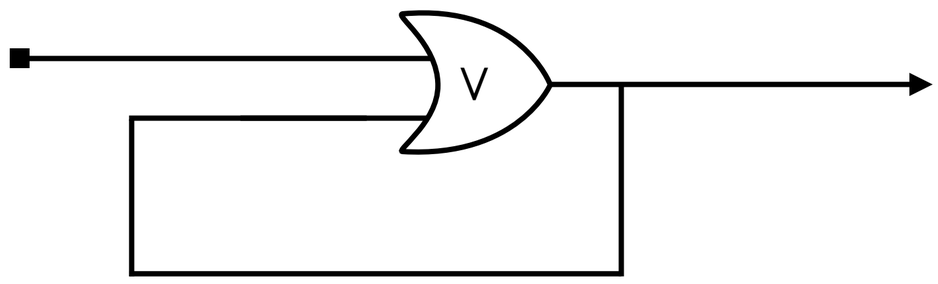
We can easily carry information forward through time in a stable medium like chalk on a chalkboard. But how could we store information in a dynamic medium like the flow of electric current, or electro-chemical interactions of neurons? How can we get the information stored in an electric current to "sit still"? We'll now explore how memory can be implemented in an electronic circuit. The details here won't matter much for the overall narrative, but if we are out to demonstrate the possibility of physical cognition, we'd better prove to ourselves first that memory itself can be physically implemented.

Warning! In this section I'm going to use the familiar-looking circuit diagrams to represent **electronic circuits**, and not **logic circuits**. This means that 1's and 0's represent states of an electric current (**on** and **off**), and lines represent **wires**, not just information pathways. In an electronic circuit, the resting state for all wires is off, so the default input signal to any logic gate is 0. Also, in an electronic circuit, you can create loops, though their behavior can be unpredictable! I've abstracted away from voltage source and ground.

The core idea is that information could be stored by locking an electrical current into a **loop**, which would sustain it over time. The circuit below embodies this idea. The signal stored in the loop is being repeatedly delivered to the output wire at right.

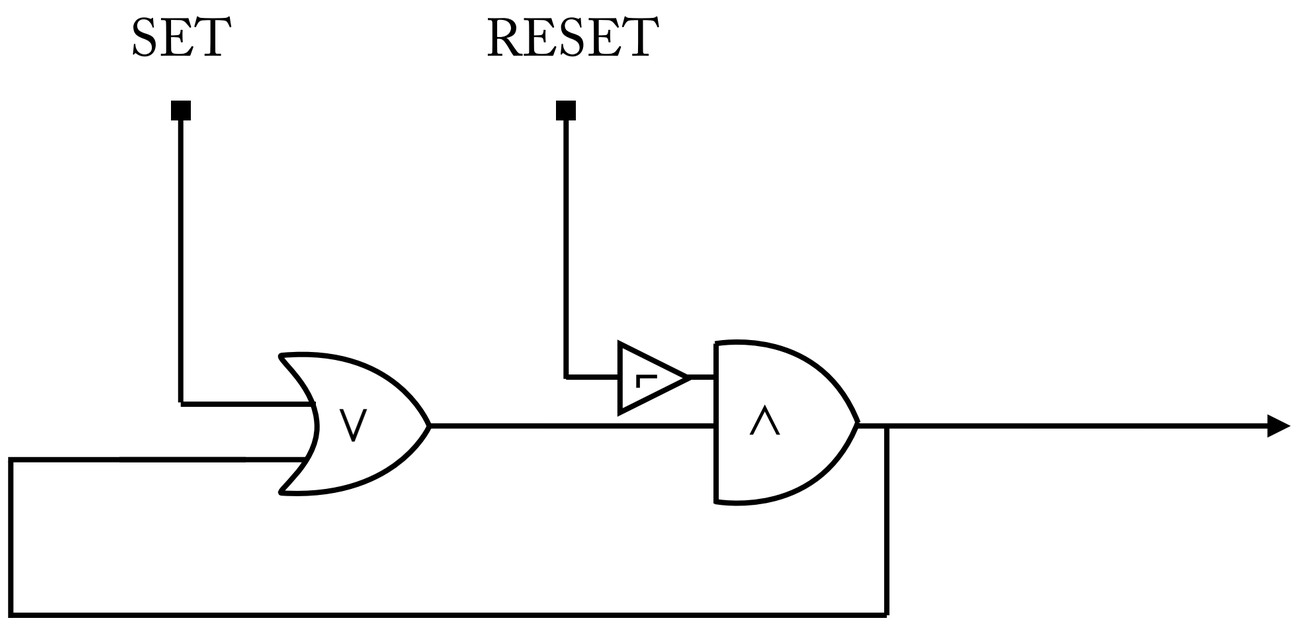


But how do we get information into the loop in the first place? A circuit like the one below does the trick. As long as the input signal is 0, the output is 0. But as soon as the circuit receives a single 1, it will output 1's forever. This is a form of memory— it carries information forward in time about whether a 1 has ever been inputted.



But it is not very useful. It's like a seesaw that always gets stuck. We want a memory that can store whatever information we like, the way a blackboard allows us to do! What we need is a way of selectively breaking the loop, and resetting the circuit at will. We want something that, at one time, can take a bit and keep reproducing it (like the circuit above), but at another time, can also start afresh if it's convenient.

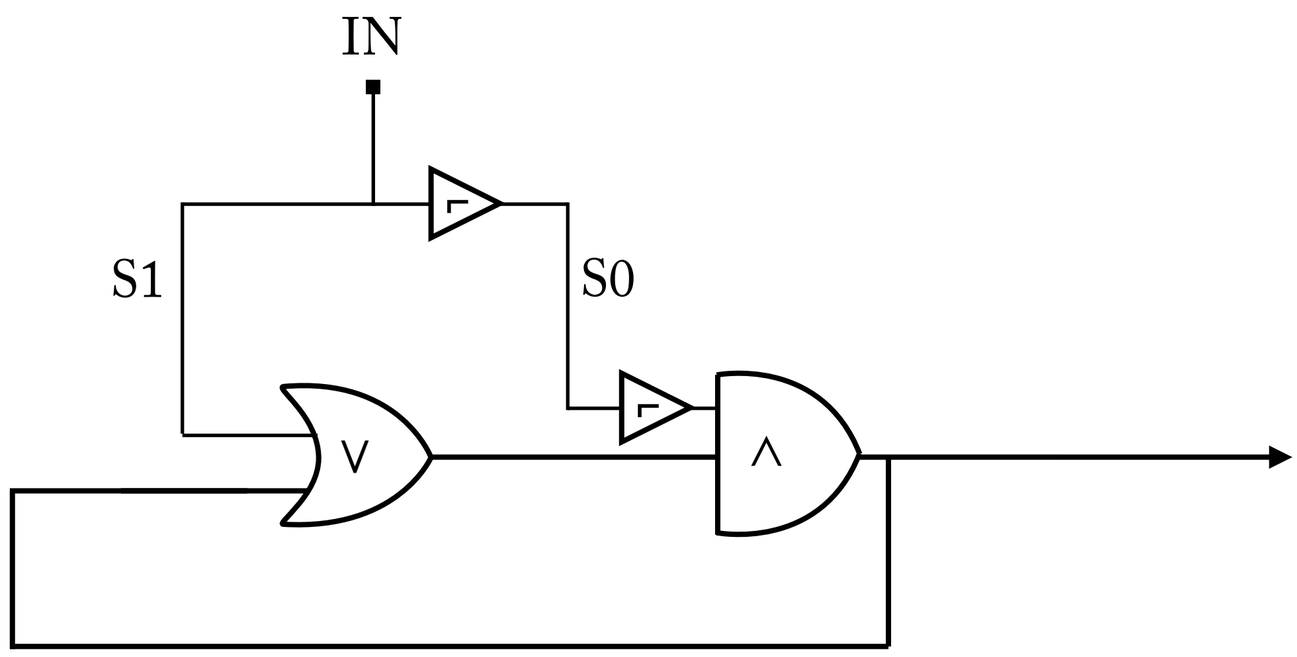
Here is an idea: introduce an AND-gate after the OR-gate. That way we can block the loop when we want to. For conceptual clarity, we'll add a NOT-gate just before the AND-gate.



To think about this circuit, let's say that, by default, RESET=0 and SET=0. In the initial "resting state", the circuit as a whole will store a 0. If at some point SET=1 for a moment, then the circuit will switch over into an "active state", in which case it stores a 1, even when SET=0 again. When you are ready to return to the resting state, let RESET=1 for a moment.

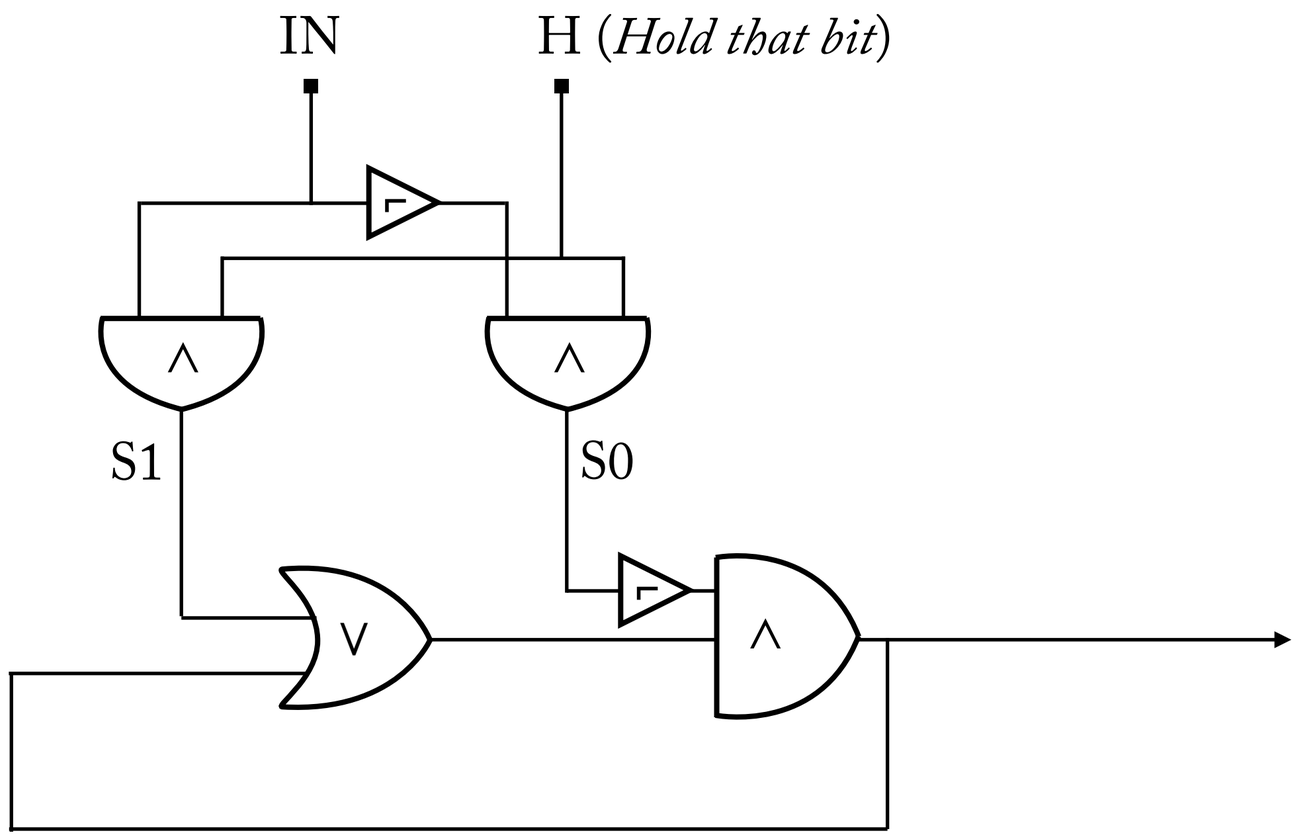
In a way, the resting state stores a 0, and the active state stores a 1. The glimmer of real memory! Can we use the two different states to store an arbitrary bit? Suppose we had 1's trigger the active state, and 0's trigger the resting state?

The natural next step is to hook up an input wire to SET and RESET (now labeled S1 and S0), so that 1's activate only the S1 side and 0's activate only the S0 side.



This is a good start, but our resulting circuit doesn't actually remember anything. The problem is that there is always *something* coming along as input (even if it's just the OFF signal).

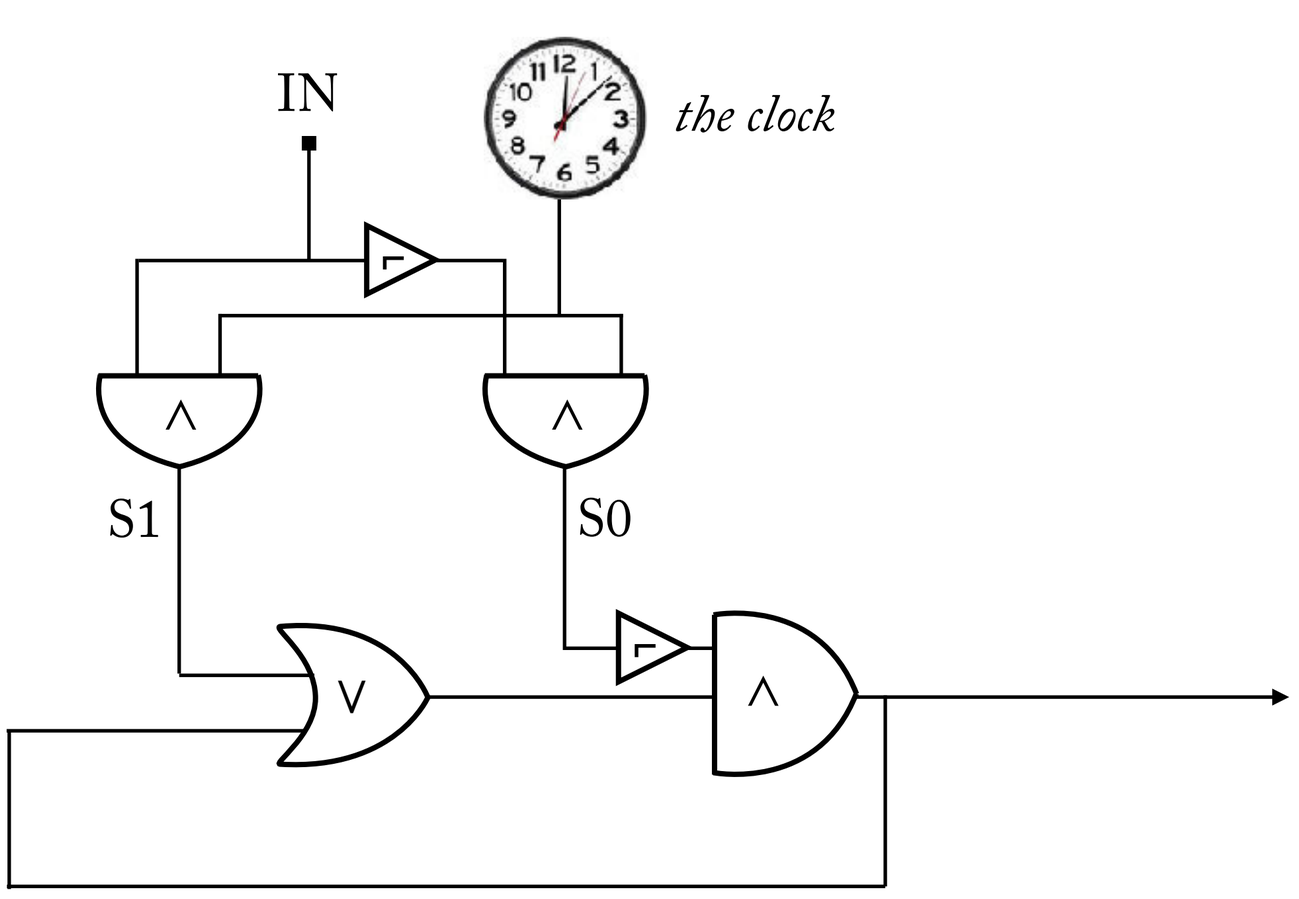
So we are going to introduce a control mechanism which we'll call "Hold that Bit" (H). The idea is that you activate H when you want to capture and store the current input bit, but if you are happy with the bit you've got, then leave H off. This circuit is known as a **flip-flop.**



Our memory is like a box. Sending a 1 through H is equivalent to opening the box; whatever the current input bit is, that bit is stored in the box. Sending a 0 to H is equivalent to closing the box; the current input bit is now irrelevant, only what's in the box matters.



There's one last step. After all, when does H get activated? Does the user somehow have to turn on each memory when they want to use it? No. For this we'll use **a clock**. A clock in this sense doesn't *store* the time, instead it marks out **ticks** at regular intervals, like a metronome. We'll connect the clock up directly to H. The idea is that, at each tick, H is activated, and the loop is either set to 1 or set to 0; after each tick, H is deactivated and the loop continues to store and output whatever bit it was last set to.



The result is what we'll call a **memory register**: a type of self-contained circuit that can store a single bit. In the future, we'll want to abstract away from all this physical circuitry, in order to focus on the logic. So I'll think of each memory as a 1-input, 1-output box which outputs a bit at the beginning of each clock cycle and inputs a new bit at the end of each clock cycle. We'll hide the input from the clock, with the assumption that the clock is hooked up in the same way to every memory register.

